Computer Networks Spring 2026
Assignment#5 (6A & 6B) Solution
Part – 1

Problem 14

a) eBGP

b) iBGP

c) eBGP

d) iBGP

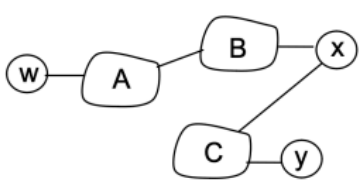
Problem 15

a) I1 because this interface begins the least cost path from 1d towards the gateway router 1c.

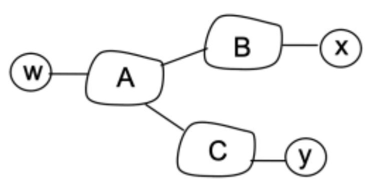
b) I2. Both routes have equal AS-PATH length but I2 begins the path that has the closest NEXT-HOP router.

c) I1. I1 begins the path that has the shortest AS-PATH.

Problem 17



X's view of the topology



W's view of the topology

In the above solution, X does not know about the AC link since X does not receive an advertised route to w or to y that contain the AC link (i.e., X receives no advertisement containing both AS A and AS C on the path to a destination.

Problem 18

BitTorrent file sharing and Skype P2P applications.
Consider a BitTorrent file sharing network in which peer 1, 2, and 3 are in stub networks W, X, and Y respectively. Due the mechanism of BitTorrent's file sharing, it is quire possible that peer 2 gets data chunks from peer 1 and then forwards those data chunks to 3. This is equivalent to B forwarding data that is finally destined to stub network Y.

Problem 19

A should advise to B two routes, AS-paths A-W and A-V.
A should advise to C only one route, A-V.
C receives AS paths: B-A-W, B-A-V, A-V.

Part – 2

Question 1:

Poisoned Reverse

To solve the count-to-infinity problem in this scenario, **Router C** must use **Poisoned Reverse**.

- **Logic:** Since Router C uses Router B to reach Router A, Router C should advertise to Router B that its cost to reach A is **infinity** (unreachable).
- **Result:** When A goes down and B updates its table, B will not try to route through C to get to A, because C has explicitly told B "I cannot reach A." This breaks the loop immediately.

Question 2:

a) **Does BGP allow Z to implement this policy?** Yes.
b) **If Yes, how? (Give at least two reasons/methods)**

BGP is a policy-based routing protocol that allows Autonomous Systems to enforce complex routing decisions based on business agreements rather than just hop counts. Z can implement this via AS-PATH Filtering: AS Z can configure an Input Policy on its link with AS Y. Z can accept route advertisements from Y only if the AS-PATH consists solely of AS Y (indicating the route originated in Y). Z will reject any advertisements where the AS-PATH contains AS X (e.g., [Y, X]). This ensures Z does not learn routes to X and will not route return traffic to X.

Question 3:

1. Usage of Attributes

- **NEXT-HOP:**
  - **Routing Table Installation:** It specifies the IP address of the router to which the packet should be forwarded to reach the destination.
  - **Recursive Lookup:** In iBGP (Internal BGP), routers use the NEXT-HOP attribute (often the IP of the edge router that learned the route) to look up the physical interface to reach that edge router via the IGP (like OSPF).
- **AS-PATH:**
  - **Loop Detection:** Before accepting a route, a router scans the AS-PATH. If it sees its own Autonomous System (AS) number in the list, it rejects the route to prevent a routing loop.
  - **Path Selection:** It is used as a metric for distance. Shorter AS-PATHs (fewer AS hops) are preferred over longer ones during the best-path selection process.

2. Implementing Policy by an Upper-Tier ISP

A network administrator implements policy primarily through **Route Filtering** and **Attribute Manipulation**:

- **Import/Export Filtering:** The admin configures rules to decide which routes to accept or advertise. For example, ensuring **Valley-Free Routing** by only advertising *Customer* routes to *Peers/Providers* (and not advertising *Peer* routes to other *Peers*).

- **Attribute Manipulation:**
    - **Local Preference:** The admin sets Local_Pref on incoming routes to control **outbound** traffic (e.g., preferring a "Customer" link over a "Provider" link to save money).
    - **AS-PATH Prepending:** The admin artificially adds their own AS number multiple times to the AS-PATH when advertising to neighbors. This makes the path look longer and less desirable, effectively controlling **inbound** traffic.

Question 4:

1. Yes. AS2 (Provider) always advertises its Customer routes (AS4) to its Providers (AS1). AS2 wants the whole world to reach its paying customer.
2. Yes. AS2 advertises Customer routes (AS4) to Peers (AS3).
3. No. Routes learned from a Peer (AS3) are only advertised to Customers. They are never advertised to Providers (AS1). If AS2 did this, AS1 would send traffic destined for AS3 through AS2, making AS2 a "free transit" service, which costs AS2 money without revenue.

Question 5:

Part (a): Dijkstra's Algorithm Table

Step-by-step computation. D(v) = current best distance to v; p(v) = predecessor. — = node finalized.

Step	N'	D(a),p(a)	D(b),p(b)	D(c),p(c)	D(d),p(d)	D(e),p(e)	D(f),p(f)	D(g),p(g)	D(t),p(t)
0	s	7, s	∞	8, s	∞	∞	∞	∞	∞
1	s, a	—	13, a	8, s	18, a	∞	∞	∞	∞
2	s,a,c	—	13, a	—	18, a	∞	14, c	15, c	∞
3	s,a,c,b	—	—	—	18, a	17, b	14, c	15, c	∞
4	…,f	—	—	—	18, a	17, b	—	15, c	∞
5	…,g	—	—	—	18, a	17, b	—	—	24, g

6	…,e	—	—	—	18, a	—	—	—	20, e
7	…,d	—	—	—	—	—	—	—	20, e
8	…,t	—	—	—	—	—	—	—	—

Part (b): Shortest Path s → t

Shortest Path	Total Cost
s → a → b → e → t	**7 + 6 + 4 + 3 = 20**

Part (c): Effect of Reducing Edge (c,f) from 6 to 2

Node f is finalized at Step 4 with D(f) = 14 via predecessor c.

At that point, g is not yet finalized. With the reduced edge (c,f) = 2:

New D(f) would be: 8 + 2 = 10 (via s → c → f)

Path s → c → f → g → t = 8 + 2 + 5 + 9 = 24

The original shortest path s → a → b → e → t (cost = 20) is unaffected by this change.

Path via f	Cost with c–f = 2	vs. Optimal
s → c → f → g → t	8+2+5+9 = 24	24 > 20 → not optimal
s → a → b → e → t	7+6+4+3 = 20	**✓ Remains shortest path**

Conclusion: The shortest path s → t does NOT change. Cost remains 20.

Question 6:

a. Initial Distance Vectors (Round 0)

Before any exchanges, each router only knows the cost of its directly connected links. The distance to itself is 0, and all other routes are ∞.

- **Router A:** [0, 1, 4, ∞, ∞, ∞]
- **Router B:** [1, 0, 2, 6, ∞, ∞]
- **Router C:** [4, 2, 0, 3, 5, ∞]
- **Router D:** [∞, 6, 3, 0, 1, 4]
- **Router E:** [∞, ∞, 5, 1, 0, 2]
- **Router F:** [∞, ∞, ∞, 4, 2, 0]

b. DV Simulation: Three Rounds

In each synchronous round, routers apply the Bellman-Ford equation to update their vectors using information received from neighbors in the *previous* round:

D_x(y) = min_v { c(x, v) + D_v(y)

Round	**D(·, A)**	**D(·, B)**	**D(·, C)**	**D(·, D)**	**D(·, E)**	**D(·, F)**
0 (init)	[0, 1, 4, ∞, ∞, ∞]	[1, 0, 2, 6, ∞, ∞]	[4, 2, 0, 3, 5, ∞]	[∞, 6, 3, 0, 1, 4]	[∞, ∞, 5, 1, 0, 2]	[∞, ∞, ∞, 4, 2, 0]
1	[0, 1, 3, 7, 9, ∞]	[1, 0, 2, 5, 7, 10]	[3, 2, 0, 3, 4, 7]	[7, 5, 3, 0, 1, 3]	[9, 7, 4, 1, 0, 2]	[∞, 10, 7, 3, 2, 0]
2	[0, 1, 3, 6, 8, 11]	[1, 0, 2, 5, 6, 9]	[3, 2, 0, 3, 4, 6]	[6, 5, 3, 0, 1, 3]	[8, 6, 4, 1, 0, 2]	[11, 9, 6, 3, 2, 0]
3	[0, 1, 3, 6, 7, 10]	[1, 0, 2, 5, 6, 8]	[3, 2, 0, 3, 4, 6]	[6, 5, 3, 0, 1, 3]	[7, 6, 4, 1, 0, 2]	[10, 8, 6, 3, 2, 0]

c. Convergence & Path Analysis

I. Has the DV protocol converged after Round 3?

No, it has not converged. To justify this, look at Router A's vector in a hypothetical Round 4. Router A calculates its distance to F by checking its neighbors B and C (using their Round 3 tables).

c) Via B: c(A,B) + D_B(F) = 1 + 8 = 9
d) Via C: c(A,C) + D_C(F) = 4 + 6 = 10
The minimum is 9. Because Router A's distance to F updates from 10 (in Round 3) to 9 (in Round 4), the routing tables are still changing, meaning convergence has not been reached.

II. State the shortest path and cost from A to F.

- **Sequence:** A -> B -> C -> D -> E -> F
- **Cost:** 1 + 2 + 3 + 1 + 2 = 9
  This matches the Bellman-Ford optimal calculation demonstrated in Round 4 above.

III. Convergence rounds vs. network diameter.

In the worst-case scenario, convergence requires **d rounds** for a network with diameter d (longest shortest path).

In this network, the shortest path between A and F spans exactly 5 links (diameter = 5). Because distance-vector protocols propagate shortest-path information one hop per round, it perfectly structurally aligns with why it requires 5 rounds for Router A to fully discover the optimal path of 9 to Router F.

d. The Count-to-Infinity Problem

If the link (D, E) degrades to 10, a pure count-to-infinity problem will **not** manifest between D and E for destination F.

- **Why?** Router E reaches F directly (cost 2) and does not rely on D. Router D also has a direct link to F (cost 4). If D's path through E suddenly costs 12, D simply checks its Bellman-Ford options and instantly switches to its direct link (4), breaking any potential loop instantly.

Before the change, D routed via E to F (D = 3), and E routed direct to F (E = 2).

4. **Round 1 (The Change):** Link (E, F) jumps to 10. Router E needs a new path. It looks at neighbor D, who claims a distance to F of 3. E does not know D's path relies on E. E happily routes via D: c(E,D) + D_D(F) = 1 + 3 = 4. E updates its distance to **4**.
5. **Round 2:** D receives E's updated vector (4). D's path to F goes through E, so D is forced to recalculate: c(D,E) + D_E(F) = 1 + 4 = 5. D updates its distance to **5**.
6. **Round 3:** E receives D's updated vector (5). E recalibrates its path via D: c(E,D) + D_D(F) = 1 + 5 = 6. E updates its distance to **6**.
7. **Round 4:** D receives E's updated vector (6). D recalibrates its path via E: c(D,E) + D_E(F) = 1 + 6 = 7. D updates its distance to **7**.

This loop continues, slowly counting up to infinity (or a predefined maximum metric) because D and E are mutually dependent on each other's poisoned information.